NOIP 2026 模拟赛（01）题解

T3 灯泡（zarowki）

【先把题目读懂】

有 n 盏灯，每盏只有亮和灭两种状态，初始由 $c_1,\dots,c_n$ 给出（$c_i=1$ 表示第 i 盏亮）。有 m 个开关，第 j 个连着第 a_j、第 b_j 两盏灯，按下时：两盏**现在状态相同**就一起取反，**状态不同**就什么都不发生。随便按，问从初始状态出发能走到多少种不同的状态，对 10^9+7 取模。

样例 1：$n=5$，状态 $1,0,1,1,0$，边是 $(1,3)$、$(3,5)$、$(2,4)$、$(1,5)$。灯 2、4 只跟对方相连，眼下又一灭一亮，那个开关永远按不动，所以这两盏从一开始就定死了。剩下 $1,3,5$ 是个三角形，能变出 4 种样子，答案是 $4\times1=4$。

规则是**带条件**的：能不能按要看当前状态，“异或线性基”那类套路直接失效。突破口是找**怎么按都不变的量**。

【分数地图】

子任务	测试点	特殊性质	分值
1	1 到 5	$n\le10$	10
2	6 到 10	图是一棵树或森林	10
3	11 到 20	图能黑白染色	20
4	21 到 30	$n\le2000$	20
5	31 到 50	没有特殊限制	40

这次考试的情况：第一档只有 7 人整档通过、8 人零散过点，第二档起没人整档通过。分水岭不是正解，而是**先把做法一的 BFS 写稳**——它是后面所有做法的对拍器，也是考场上最该先到手的 10 分。

【做法一：把所有能到的状态走一遍（10分）】

$n\le10$ 时状态最多 $2^{10}=1024$ 种，BFS 全走一遍：状态压成一个整数（第 i 位为 1 表示第 i 盏亮），每次枚举 m 个开关，能按就按出下一个状态，没访问过就入队。

```
#include <bits/stdc++.h>
using namespace std;

int n, m;
int ea[1005], eb[1005];     // 第 i 个开关连接 ea[i] 和 eb[i] 两盏灯
int start[15];              // 一开始每盏灯是开还是关
```

```
bool vis[1 << 12];          // 这个状态有没有到过

int main() {
    scanf("%d %d", &n, &m);
    for (int i = 0; i < n; i++) {
        scanf("%d", &start[i]);
    }
    for (int i = 0; i < m; i++) {
        scanf("%d %d", &ea[i], &eb[i]);
        ea[i]--;                // 题目从 1 开始编号，我们改成从 0 开始
        eb[i]--;
    }

    // 把初始状态压缩成一个整数：第 i 位是 1 就表示第 i 盏灯亮着
    int s = 0;
    for (int i = 0; i < n; i++) {
        if (start[i] == 1) {
            s = s + (1 << i);
        }
    }

    // 从初始状态出发，把所有能走到的状态都走一遍
    queue<int> q;
    vis[s] = true;
    q.push(s);

    int ans = 0;
    while (!q.empty()) {
        int now = q.front();
        q.pop();
        ans++;                  // 每走出一个状态，答案就多一种

        for (int i = 0; i < m; i++) {
            int x = (now >> ea[i]) & 1;
            int y = (now >> eb[i]) & 1;
            if (x != y) {
                continue;       // 两盏灯状态不同，按这个开关没用
            }
            int nxt = now ^ (1 << ea[i]) ^ (1 << eb[i]);
            if (vis[nxt] == false) {
                vis[nxt] = true;
                q.push(nxt);
            }
        }
    }
    printf("%d\n", ans);
    return 0;
}
```

三个要点：`x != y` 时 `continue`，对应“状态不同按了没用”；`now ^ (1 << ea[i]) ^ (1 << eb[i])` 用异或把两位同时取反；`vis` 漏标会死循环。零散过点的 8 位同学，错因基本就两种——位没对齐（`1 << i` 和灯号差一位）、漏标 `vis`，造个 $n = 3$ 的数据手算一遍就能查出来。

【分层：给每盏灯编个层号】

状态多达 2^n 种，硬搜没戏，得抓“什么量被锁住了”。抓手是**分层**。

对**每个连通块**分别做（图不连通，各块互不相干，最后答案相乘）：从块内任取一盏灯当起点，层号 0；它的邻居层号 1；再往外层号 2……层号就是“到起点的最短距离”，一遍 BFS 算完。层号是偶数的叫**黑层**，是奇数的叫**白层**。

这样分层有个硬性质：每条边两端的层号最多差 1，所以**每条边两端要么一黑一白**（层号差 1），**要么同色**（层号相同）。记住这句，下面全靠它。

【矛盾点】

分层等于给每盏灯规定了一个“该有的状态”：黑层该灭、白层该亮。

实际状态跟它不一致的灯，就叫一个**矛盾点**，个数记作 w。

为什么这么定义？因为一条边两端一黑一白，两盏灯的状态搭配只有 4 种，数一下矛盾点：

两端状态（黑，白）	矛盾点在哪	个数	按得动吗	按完
灭，灭	白那头	1	能动	矛盾点跑到黑那头
亮，亮	黑那头	1	能动	矛盾点跑到白那头
亮，灭	两头都是	2	不能	—
灭，亮	两头都不是	0	不能	—

四行看完，一句话：**两端一黑一白时，恰有 1 个矛盾点 $\Longleftrightarrow$ 这个开关按得动**；按下去矛盾点总数不变，只是**从边的一头挪到另一头**。没有奇环的块不会有同层边（有的话，树上那条路加它就凑成奇环），每条边都一黑一白，所以每个开关都适用。

（注意“矛盾点”跟“亮着的灯”是两回事：黑层亮、白层灭都算矛盾点，同一个块里亮灯数和矛盾点数并不相同。）

于是在没有奇环的块里，整道题变成一道挪棋子的问题：w 个矛盾点摆在灯上，每次把一个挪到相邻的空位，问能摆出多少种样子。

【矛盾点能挪到任意一个空位上】

想一想：块里的矛盾点，能不能挪到指定的空点（没有矛盾点的灯）上去？能。块是连通的，让那个空点沿路一步步挪向这枚矛盾点：前面挡着一枚矛盾点，就把它挪进空点、空点前移一格；前面本来就是空灯，空点直接走过去。等空点挪到这枚矛盾点脚下，效果就等于把它搬到了原来那个空点上。每一步挪进去的都是刚空出来的格子，不会卡；反过来从矛盾点那头往外推，前面堵着一枚，卡住。

既然有空点就能去，那**矛盾点个数相同的两种摆法一定能互达**：把一枚“现在有、目标里没有”的挪到一盏“目标里有、现在没有”的灯上（后者眼下空着，挪得动），每做一次就离目标近一步。w 个矛盾点放进 k 盏灯里，方案数就是 $C(k,w)$。

样例 2 验一下：6 盏灯的链，$0,1,0,1,0,1$ 分层后是黑、白、黑、白、黑、白，正好黑层全灭、白层全亮，$w=0$，$C(6,0)=1$，跟“一次都按不动”吻合。

【有奇环的块】

分层照做，但奇环会来捣乱。**假如**奇环上每条边两端的层号都差 1，那么沿环走一步，层号就 ±1；走完整整**奇数**步，奇数个 ±1 加起来一定不是 0，可回到起点层号必须不变——矛盾。所以环上**必有一条边两端层号相同**。

这条边两端同色：一亮一灭时按不动，w 不变；按得动时两枚矛盾点同进同出，要么一次多 2 个，要么一次少 2 个。

两件事分开记：

- 其余边（两端一黑一白）：w 不变，同 w 的摆法随便换；
- 同层边：w 加 2 或减 2，一路 ±2 推下去，0 到 k 之间与初始那个 w 同奇偶的值全都能取到（中途按不动时，先用流通把两端调成同状态再按）。

合起来：**凡 w 与初始同奇偶的摆法，全都能到**。再数个数，两头一夹：

- 至少 2^{k-1} 种：w 取遍与初始同奇偶的值，每个值下的摆法都能到。而“哪几盏是矛盾点”和“灯是什么状态”是一一对应的（黑层亮的、白层灭的正好是矛盾点，反过来也就定了每盏的亮灭），所以 2^k 种摆法就是全部状态，w 奇偶各占一半。
- 至多 2^{k-1} 种：按一次翻两盏，亮灯数的奇偶在哪种图里都守恒，所以能到的状态超不过“亮灯数与初始同奇偶”的那些，也恰好这么多。

有奇环的块：能到的恰好是亮灯数与初值同奇偶的状态，共 2^{k-1} 种。

拿三角形验：3 盏灯从全灭出发，$2^{3-1}=4$ 种。同样的三个点排成链就只有 3 种（$w=1$ 守恒，$C(3,1)=3$）：从 000 只能到 110 和 011，而这两个点除了按回 000 再

无别的出口，`101` 根本摸不到——想让灯 2 的亮“传”给灯 3，按 $(2,3)$ 时一亮一灭，堵死。差的就是那条同层边。

【正解：把各块拼起来（100分）】

每个连通块单独算，结果相乘：**没有奇环的块**贡献 $C(k,w)$；**有奇环的块**贡献 2^{k-1}。

一遍 BFS 全干完：顺手分层，撞见两端同层的边就说明这块有奇环，同时数出点数 k 和矛盾点数 w。代码里层号模 2 存在 `color[]`；`canTwoColor` 变成 `false`，就是这块撞上了同层边、有奇环。组合数用“阶乘 + 逆元”预处理后 $O(1)$ 查询。

```
#include <bits/stdc++.h>
using namespace std;

const long long MOD = 1e9 + 7;  // 模数。1e9+7 比 1000000007 好读，也不容易抄错
const int MAXN = 2e5 + 5;       // n 最大 20 万，多开 5 格保险

int n, m;
int light[MAXN];            // 每盏灯一开始是亮还是灭
int color[MAXN];            // 染色结果：0 或 1，-1 表示还没染
vector<int> edge[MAXN];     // 每盏灯连着哪些灯

long long fact[MAXN];       // fact[i] = i!
long long invfact[MAXN];    // invfact[i] = i! 的逆元

// 快速幂：算 a 的 b 次方，边乘边取模
long long power(long long a, long long b) {
    long long res = 1;
    while (b > 0) {
        if (b & 1) {
            res = res * a % MOD;
        }
        a = a * a % MOD;
        b = b >> 1;
    }
    return res;
}

int main() {
    scanf("%d %d", &n, &m);
    for (int i = 1; i <= n; i++) {
        scanf("%d", &light[i]);
    }
    for (int i = 0; i < m; i++) {
        int x, y;
        scanf("%d %d", &x, &y);
        edge[x].push_back(y);
```

```
    edge[y].push_back(x);
}

// 先把阶乘和阶乘的逆元都算好，后面求组合数用
fact[0] = 1;
for (int i = 1; i <= n; i++) {
    fact[i] = fact[i - 1] * i % MOD;
}
invfact[n] = power(fact[n], MOD - 2);
for (int i = n; i >= 1; i--) {
    invfact[i - 1] = invfact[i] * i % MOD;
}

for (int i = 1; i <= n; i++) {
    color[i] = -1;
}

long long ans = 1;
for (int start = 1; start <= n; start++) {
    if (color[start] != -1) {
        continue;               // 这一块前面已经处理过了
    }

    // 从 start 出发，把连在一起的这一整块都染上色
    vector<int> nodes;          // 记下这一块有哪些灯
    queue<int> q;
    color[start] = 0;
    q.push(start);
    bool canTwoColor = true;    // 这块能不能黑白染色

    while (!q.empty()) {
        int u = q.front();
        q.pop();
        nodes.push_back(u);
        for (int i = 0; i < (int)edge[u].size(); i++) {
            int v = edge[u][i];
            if (color[v] == -1) {
                color[v] = color[u] ^ 1;   // 染成相反的颜色
                q.push(v);
            } else if (color[v] == color[u]) {
                canTwoColor = false;       // 相邻两灯同色，说明这一块有奇环
            }
        }
    }

    int k = (int)nodes.size();
    if (canTwoColor == false) {
        // 有奇环：这一块的贡献是 2 的 (k-1) 次方
        ans = ans * power(2, k - 1) % MOD;
    } else {
        // 能黑白染色：数一数有几盏灯的开关状态和染的颜色对不上
```

```
            int bad = 0;
            for (int i = 0; i < k; i++) {
                int v = nodes[i];
                if (light[v] != color[v]) {
                    bad++;
                }
            }
            // 这一块的贡献是从 k 个位置里选 bad 个：C(k, bad)
            ans = ans * fact[k] % MOD;
            ans = ans * invfact[bad] % MOD;
            ans = ans * invfact[k - bad] % MOD;
        }
    }
    printf("%lld\n", ans);
    return 0;
}
```

三个要点：常量写 `1e9 + 7`、`2e5 + 5`，别抄一串零（多一个少一个就是 0 分，还查不出来）；`invfact` 先求最后一个再倒推，是“一排阶乘逆元”的标准写法，`power` 顺手也算 2^{k-1}；每个点每条边各访问一次，加预处理共 $O(n+m)$。

【新手容易踩的坑】

1. **只从 1 号灯分层，或者忘了把各块乘起来**。图不连通，每个块都要各选起点各算一遍，答案是各块答案的**乘积**。
2. **拿 $C(k,w)$ 去算有奇环的块**。那里的答案只由 k 决定，是 2^{k-1}。
3. **担心换起点会不会算错**。不会：换起点相当于把黑白整体调换，w 变成 $k-w$，而 $C(k,w)=C(k,k-w)$，答案不变。

【这道题带走了什么】

- **操作带条件时，先找个量把状态“翻译”一遍**。分层之后，“能不能按”变成“两端矛盾点是不是恰有一个”（同层边除外），题目基本退化成挪棋子。
- **判奇环不用单独写**。分层时撞见同层边就是奇环，分层、判环、数 w 一趟 BFS 全出来。
- **难的那一半，砍掉一些边变简单、再把边放回去**。奇环块 = 生成树 + 多出来的同层边：树的结论照用，这些边负责给 w 打开 ± 2 的口子。